

dotnet/designs: accepted/2020/better- obsoletion/binaryformatter-obsoletion.md

dotnet/designs: accepted/2020/better-obsoletion/binaryformatter-obsoletion.md

- dotnet, GitHub.

- Published: date not stated
- Original: <https://github.com/dotnet/designs/blob/main/accepted/2020/better-obsoletion/binaryformatter-obsoletion.md>
- Preserved from: <https://github.com/dotnet/designs/blob/main/accepted/2020/better-obsoletion/binaryformatter-obsoletion.md> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

binaryformatter-obsoletion.md

dotnet/designs at main, path accepted/2020/better-obsoletion/binaryformatter-obsoletion.md.

BinaryFormatter Obsoletion Strategy

Owner [Levi Broderick](#)

This document applies to the following framework versions:

- .NET 5.0+

Applications which target .NET Framework 2.x, .NET Framework 4.x, or .NET Core 3.1 are *not affected* by this proposal. `BinaryFormatter` will continue to work as expected in those applications. Consumers of `BinaryFormatter` are still advised to consult the [BinaryFormatter security guide](#) to help inform their risk assessment.

Introduction

As part of modernizing the .NET development stack and improving the overall health of the .NET ecosystem, it is time to sunset the `BinaryFormatter` type. `BinaryFormatter` is the mechanism by which many .NET applications find themselves exposed to critical security

vulnerabilities, and its continued usage results in numerous such incidents every year across both first-party and third-party code.

We have messaged previously that `BinaryFormatter` is not safe for untrusted input, but outside of the security community this guidance hasn't really taken root. This is partly because there's already a critical mass of code using `BinaryFormatter`, and developers are generally disincentivized from changing existing code. It's also partly due to the fact that threat modeling is an advanced topic not well understood by the majority of developers. Without the .NET team introducing a forcing function, the ecosystem holistically does not feel the need to improve.

See the [BinaryFormatter security guide](#) for more information on the problem space.

`BinaryFormatter` also leads to the creation of fragile and non-versionable types. Its operation relies primarily on private reflection over an object's instance fields. This ties the serialized payload format to internal implementation details of the target types. Updating or improving the implementation details of the target type often creates compatibility problems between serialized versions.

Encouraging migration away from `BinaryFormatter` has the additional benefit of making applications more friendly to linker trimming. .NET is [heavily invested in linker trimming](#) as a means of reducing the deployed footprint of .NET-based applications and web services. This capability relies upon the compiler being able to statically analyze what code paths are reachable from the application. `BinaryFormatter` and other serializers which perform unbounded reflection based on input from the wire are not conducive to this goal, as the real type information isn't known until the `Type.GetType` call at runtime. We would like to move these applications onto other serializers (such as those in [the `\_System.Text.Json` package](#)) to help drive our ability to enable trimming throughout the ecosystem.

Timeline overview

.NET 5 (Nov 2020)

- Allow disabling `BinaryFormatter` via an opt-in feature switch
 - ASP.NET projects disable `BinaryFormatter` by default but can re-enable
 - WASM projects disable `BinaryFormatter` with no ability to re-enable
 - All other project types (console, WinForms, etc.) enable `BinaryFormatter` by default
- .NET produces guidance document on migrating away from `BinaryFormatter`
- All outstanding `BinaryFormatter`-related issues resolved *won't fix*
- Introduce a `BinaryFormatter` tracing event source
- `Serialize` and `Deserialize` marked obsolete *as warning*

.NET 6 (Nov 2021)

- No new `[Serializable]` types introduced
- No new calls to `BinaryFormatter` from any first-party dotnet org code base

- All first-party dotnet org code bases begin migration away from `BinaryFormatter`

.NET 7 (Nov 2022)

- All first-party dotnet org code bases continue migration away from `BinaryFormatter`
- References to `BinaryFormatter` APIs marked obsolete as *warnings* in .NET 5 now result in build *errors*
 - A back-compat switch is made available to turn these back to warnings

.NET 8 (Nov 2023)

- All first-party dotnet org code bases complete migration away from `BinaryFormatter`
- `BinaryFormatter` disabled by default across all project types
- All not-yet-obsolete `BinaryFormatter` APIs marked obsolete as *warning*
- Additional legacy serialization infrastructure marked obsolete as *warning*
- No new `[Serializable]` types introduced (all target frameworks)

.NET 9 (Nov 2024)

- Remainder of legacy serialization infrastructure marked obsolete as *warning*
- `BinaryFormatter` infrastructure removed from .NET
 - Back-compat switches also removed

Timeline specifics

Allow disabling `BinaryFormatter` via an opt-in feature switch (.NET 5)

In .NET 5 we will introduce a [feature switch](#) to selectively allow `BinaryFormatter` to be disabled in an application. This can be accomplished in one of three ways, as demonstrated below.

Disabling `BinaryFormatter` via the application's `.csproj`:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net5.0</TargetFramework>

    <EnableUnsafeBinaryFormatterSerialization>>false</EnableUnsafeBinaryFormatterSerialization>
  </PropertyGroup>
</Project>
```

Disabling `BinaryFormatter` via the application's *runtimeConfig.template.json*:

```
{
  "configProperties": {
    "System.Runtime.Serialization.EnableUnsafeBinaryFormatterSerialization": false
  }
}
```

Disabling `BinaryFormatter` via an explicit call to `AppContext.SetSwitch`:

```
AppContext.SetSwitch("System.Runtime.Serialization.EnableUnsafeBinaryFormatterSerializati
on", false);
```

If this switch is set to *true*, `BinaryFormatter` serialization + deserialization will be enabled for the application. If this switch is set to *false*, `BinaryFormatter` serialization + deserialization will be disabled for the application and will throw `NotSupportedException`. Setting the value to *false* also allows the .NET linker to trim away `BinaryFormatter`-related code paths, reducing the total disk footprint of the resulting binary.

If the switch is not set, the target project type will determine whether `BinaryFormatter` serialization + deserialization is allowed or disallowed. Note that this is determined by the application type. Libraries cannot set this switch via their own *.csproj* or *runtimeConfig.json* files.

We expect high-sensitivity applications to set this switch to **false** as part of an overall attack surface reduction strategy. Any document we produce that details how to configure high-sensitivity applications should recommend setting this switch. The scenario for setting this switch to false eagerly is that the developer does not expect the application to invoke `BinaryFormatter`-related code paths during normal operations, and they want some extra assurance that a latent bug or untested edge case in a dependency cannot re-activate the `BinaryFormatter` infrastructure.

Certain project types - such as WinForms projects - may use `BinaryFormatter` as an implementation detail. Disabling `BinaryFormatter` for these project types in .NET 5 may interfere with the correct functionality of the application. See later in this document for the approach the runtime will take for ensuring this switch works correctly in future .NET versions.

Disabling `BinaryFormatter.Serialize` and `Deserialize` via the feature switch doesn't affect the behavior of any other APIs on `BinaryFormatter`. For instance, the existing `BinaryFormatter` property getters and setters will continue to work. Types like `ObjectManager` which are intended to support the `BinaryFormatter` infrastructure will also work if the application has direct calls into those APIs. If the application does not have direct calls to these APIs, the linker will likely prune these now-unused types when it prunes the `BinaryFormatter` implementation.

There is no API to query whether a call to `BinaryFormatter.Serialize` or `BinaryFormatter.Deserialize` will fail due to this switch being set. Callers can instead catch the `NotSupportedException` and query the `AppContext` manually to see if support for serialization has been disabled. An example of this is provided below.

```
BinaryFormatter formatter = new BinaryFormatter();
try
{
    formatter.Serialize(/* ... */);
}
catch (NotSupportedException ex)
{
    if
    (AppContext.TryGetSwitch("System.Runtime.Serialization.EnableUnsafeBinaryFormatterSeriali
zation", out bool isEnabled)
    && !isEnabled)
    {
        // BinaryFormatter serialization support is disabled; handle it here
    }
    else
    {
        throw; // BinaryFormatter serialization is enabled; rethrow exception
    }
}
```

The libraries team will also document known scenarios that will be broken when the `BinaryFormatter` code paths are disabled. One such example is attempting to read `.resx` files with embedded `BinaryFormatter`-serialized objects. These scenarios are not expected to be common in web applications, but they do nevertheless exist to a limited extent.

Disablement of `BinaryFormatter` for certain project types (.NET 5)

For WASM (Blazor Client) projects, the `BinaryFormatter` feature switch will be hardcoded to **disabled**. The implementations of `BinaryFormatter.Serialize` and `BinaryFormatter.Deserialize` will throw `NotSupportedException`. There is no ability for WASM applications to re-enable this feature through configuration or through a runtime switch. This hardcoded behavior extends to all other novel application types that might be introduced after the .NET 5 timeframe, including mobile apps and other restricted environment configurations.

Starting with ASP.NET 5.0, the implementations of `BinaryFormatter.Serialize` and `BinaryFormatter.Deserialize` will *by default* throw `NotSupportedException`. This is being done to reduce the attack surface area of web applications. This project type has historically been the largest target of deserialization exploits.

To re-enable the `BinaryFormatter` code paths in ASP.NET projects, the application must set the feature switch described above in their `.csproj`, and as demonstrated below.

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net5.0</TargetFramework>
    <!-- WARNING: re-enabling the switch below could subject your application to critical
    security vulnerabilities -->

    <EnableUnsafeBinaryFormatterSerialization>true</EnableUnsafeBinaryFormatterSerialization>
  </PropertyGroup>
</Project>
```

For other project types - such as console, WinForms, and WPF - `BinaryFormatter` will continue to be enabled by default. No configuration is necessary if the app wishes to continue using the `BinaryFormatter` APIs. However, developers can still disable `BinaryFormatter` by using the feature switch mechanism mentioned previously.

.NET produces guidance document on migrating away from BinaryFormatter (.NET 5)

The .NET team will produce a document showing how to accomplish various serialization scenarios in manners that don't involve `BinaryFormatter`. Some examples of scenarios that we can show:

- Reading the raw binary contents of a graphic and constructing an Image from it.
- Using `System.Text.Json` to serialize / deserialize object models, including dictionaries and other collection types.
- Demonstrating how to serialize / deserialize `CultureInfo` via just the culture name.
- Transmitting `Exception` objects safely across the wire.

We need to be prepared for the fact that not every scenario will be achievable. For example, .NET Framework allows serializing delegates, but .NET Core doesn't. This is by design and no replacement is intended. As another example, `System.Text.Json` doesn't allow deserializing cyclic object graphs by default. We would demonstrate how to opt in to these features while calling attention to the fact that consumers should consider the security impact of doing so.

We are also intentionally *not* providing sample code on the use of a `SerializationBinder` as a security mechanism, as we cannot guarantee the safety of this design and should not promote its usage as such.

All outstanding BinaryFormatter-related issues resolved won't fix (.NET 5)

We'll perform a final pass through all the `BinaryFormatter`-related issues opened on GitHub, resolving all of them *won't fix* and pointing them to this document. There are only three exclusions to this pass:

- Functional regressions introduced as part of an in-place servicing release.
- Severe performance regressions introduced as part of an in-place servicing release.
- Open issues which track implementing this deprecation plan.

All other issues, including incompatibility between .NET Framework + .NET Core, or asking for new types to be marked `[Serializable]`, or asking for new features or bug fixes, will be closed.

Introduce a `BinaryFormatter` tracing event source (.NET 5)

To help app developers detect unexpected calls to `BinaryFormatter` within dependencies, we will enable a tracing event source within `BinaryFormatter` itself. This tracing source will provide notification that `BinaryFormatter.Serialize` or `BinaryFormatter.Deserialize` was called, and developers can use this information as part of their own process of auditing `BinaryFormatter` usage.

It is anticipated that this mechanism will use the `System.Diagnostics.DiagnosticSource` infrastructure. More information on using this infrastructure is available at [the user guide](#).

App developers can test-run their application with this trace listener enabled, and if they see no unexpected calls to `BinaryFormatter` they can confidently set the `AppContext` setting to disable `BinaryFormatter` in production. This allows them to limit their exposure to `BinaryFormatter`-related vulnerabilities within their own services.

Serialize and Deserialize marked obsolete as warning (.NET 5)

The `Serialize` and `Deserialize` methods on `BinaryFormatter`, `Formatter`, and `IFormatter` will be marked obsolete *as warning*, as shown below.

```

namespace System.Runtime.Serialization.Formatters.Binary
{
    public sealed class BinaryFormatter : IFormatter
    {
        [Obsolete("...", error: false, DiagnosticId = "...")]
        public object Deserialize(Stream serializationStream);
        [Obsolete("...", error: false, DiagnosticId = "...")]
        public void Serialize(Stream serializationStream, object graph);
    }
}

namespace System.Runtime.Serialization
{
    public interface IFormatter
    {
        [Obsolete("...", error: false, DiagnosticId = "...")]
        object Deserialize(Stream serializationStream);
        [Obsolete("...", error: false, DiagnosticId = "...")]
        void Serialize(Stream serializationStream, object graph);
    }

    public abstract class Formatter : IFormatter
    {
        [Obsolete("...", error: false, DiagnosticId = "...")]
        public abstract object Deserialize(Stream serializationStream);
        [Obsolete("...", error: false, DiagnosticId = "...")]
        public abstract void Serialize(Stream serializationStream, object graph);
    }
}

```

The error message text is TBD but will direct callers to the documentation available at <https://aka.ms/binaryformatter>. The diagnostic id will be set per the [better obsolescence](#) specification.

This is likely to be very noisy since it's the first change that will begin to produce warnings in our developer audience's own code. We will continue updating the guidance document beyond the .NET 5.0 timeframe so that it's clear what kinds of changes developers should make within their own code bases.

The behavior of the obsoleted APIs will not change. Applications which suppress or ignore the warning will see no runtime behavioral difference, assuming the "disable `BinaryFormatter` entirely" feature switch mentioned previously in this document has not been utilized.

No new `[Serializable]` types introduced (.NET 6)

.NET 5.0 will be the final version of .NET that introduces `[Serializable]` types. Beginning with .NET 6.0, no new types will be marked `[Serializable]`, nor will the `[Serializable]` attribute be added to existing types or to types ported from .NET Framework. Introducing serializable annotations runs counter to our goal of spinning down usage of `BinaryFormatter` framework-wide.

This prohibition on new serializable types applies only to projects in the *dotnet* org on GitHub. Other frameworks, libraries, or applications built on .NET may continue to use `[Serializable]` in their own code bases.

The exclusion is for dotnet org first-party projects which target *netstandard2.0* or earlier. For these projects, `Exception`-derived types should continue to be annotated as `[Serializable]` so that they may be remoted across app domain boundaries. Any new `Exception`-derived types that are defined in a library which is not available to *netstandard2.0* callers will not be annotated with `[Serializable]`. We should be following our own guidance on how to safely serialize exception information without involving `BinaryFormatter`.

No new calls to `BinaryFormatter` from any first-party dotnet org code base (.NET 6)

Beginning with .NET 6.0, no *new* code introduced into the *dotnet* org GitHub repos may call into `BinaryFormatter`, *regardless of target runtime*. Introducing such code would run counter to our goal of spinning down usage of `BinaryFormatter` framework-wide. This prohibition does not apply to user or partner code. Those code bases may continue to use `BinaryFormatter` as needed, though they are strongly discouraged from doing so.

This should not prevent us from refactoring existing first-party code which uses `BinaryFormatter`. The intent of this policy is to prevent `BinaryFormatter`'s usage in new scenarios, not to prevent teams from maintaining their own code bases. Documentation, tests, and other "non-shipping" code is exempt.

- All first-party dotnet org code bases begin migration away from `BinaryFormatter` (.NET 6)

The WinForms designer should enable new .resx formats which do not involve `BinaryFormatter`, following our guidance on how to serialize such resources safely. These new .resx formats should be the default for any scenario which previously required `BinaryFormatter` involvement. MSBuild and other components which handle `BinaryFormatter`-serialized payloads opaquely should continue to process these payloads. No action is required on their end unless they attempt to deserialize these payload.

If `BinaryFormatter` is used in any other data persistence scenario (e.g., clipboard or app saved state), the area owners should consult the guidance document to create a safe alternative. All code paths which involve `BinaryFormatter` must have equivalent non-`BinaryFormatter` alternatives available, though these alternatives needn't have 100% feature parity. Area owners should make every effort to enable the non-`BinaryFormatter`

code paths as the default for creating new data. It remains allowable for these libraries to read existing data via `BinaryFormatter`.

Auditing first-party code bases for `Type.GetType` calls

As part of this, dotnet org developers should audit the area of ownership for calls to `Type.GetType` and related APIs, as these could function as unrestricted deserialization equivalents. Any call sites which deal with untrusted data should clearly be annotated as such in code comments. If necessary, the owners should open issues to track removing the dangerous code patterns or documenting the API as dangerous. This will allow us to make more informed estimates of our libraries' own exposure to potential RCE attacks.

APIs previously marked obsolete (*as warning*) in .NET 5 now produce build *errors* by default (.NET 7)

In .NET 5, we marked several `BinaryFormatter`-related APIs obsolete as warning (see [the breaking change notice](#)). In .NET 7, references to these APIs will produce build *errors*. This will cause compilation failures in applications which directly consume these APIs, and developers who call these APIs must take action in order to migrate their applications onto .NET 7.

An opt-in back-compat switch will be made available for applications which cannot migrate away from `BinaryFormatter` in the .NET 7 timeframe. We will utilize the same compat switch that was introduced in .NET 5.

```
<Project>
  <PropertyGroup>
    <TargetFramework>net7.0</TargetFramework>
    <!-- Setting the switch below turns the errors back to warnings. -->
    <!-- WARNING: re-enabling the switch below could subject your application to critical
security vulnerabilities. -->

    <EnableUnsafeBinaryFormatterSerialization>true</EnableUnsafeBinaryFormatterSerialization>
  </PropertyGroup>
</Project>
```

All first-party dotnet org code bases complete migration away from `BinaryFormatter` (.NET 8)

No first-party *dotnet* org code base contains an enabled-by-default calls to `BinaryFormatter`. The call sites may remain in the library but must be guarded by an opt-in switch. This opt-in switch must be specified by *the app developer*, not by an "I'm serialized using `BinaryFormatter`" flag within the payload itself. All other payload formats must be written or read using a `BinaryFormatter` alternative.

Importantly, none of our project types (WinForms / WPF / ASP.NET / mobile / etc.) can require developers to opt back in to allowing `BinaryFormatter` for any greenfield

development. Applications which require `BinaryFormatter` for app-compat reasons must require developers to perform an explicit opt-in gesture to re-enable these code paths.

BinaryFormatter disabled by default across all project types (.NET 8)

Beginning with .NET 8.0, the "disable `BinaryFormatter`" feature switch is activated by default across all project types. Developers must perform an explicit opt-in gesture to re-enable these code paths. This will utilize the same compat switch introduced in .NET 5.

New obsoletions in .NET 8

In .NET 8.0, the following APIs will be marked `[Obsolete]` *as error* by default. These will utilize the existing `SYSLIB0011` warning code. The errors can be converted back to warnings by using the `<EnableUnsafeBinaryFormatterSerialization>` compat switch within the project file.

- The entirety of the `BinaryFormatter` type.
- The entirety of the `IFormatter` interface.
- The entirety of the `Formatter` type.

The following APIs will be marked `[Obsolete]` *as warning* using a new warning code (tentatively `SYSLIB0050`).

Newly obsolete types

- `IFormatterConverter` interface and `FormatterConverter` concrete type
- `FormatterServices` static class
- `IObjectReference` interface
- `ISurrogateSelector` and `ISerializationSurrogate` interfaces, and the `SurrogateSelector` concrete type
- `ISafeSerializationData` and `SafeSerializationEventArgs`
- `StreamingContextStates` enum
- `ObjectIDGenerator`
- `ObjectManager`
- `SerializationObjectManager`
- `FormatterTypeStyle`, `FormatterAssemblyStyle`, and `TypeFilterLevel` enums
- `IFieldInfo` interface

(All APIs which expose these types through their public API surface would also be obsoleted.)

Newly obsolete members on existing types

- `Type.IsSerializable` public property (and all overridden implementations)
- `TypeAttributes.Serializable` enum value
- `FieldInfo.IsNotSerialized` public property
- `FieldAttributes.NotSerialized` enum value

- `ISerializable.GetObjectData` method, but not the type itself
- `SerializationInfo` and `StreamingContext`, all ctors, but not the types themselves

The following APIs will be marked `[Obsolete]` as *warning* using a new warning code (tentatively `SYSLIB0051`) and marked `[EditorBrowsable(Never)]`.

- All externally visible (public or protected) serialization ctors: `.ctor(SerializationInfo, StreamingContext)`.
- All public (not explicit or protected) implementations of `IObjectReference.GetRealObject`.
- All public (not explicit or protected) implementations of `ISerializable.GetObjectData`.

The following APIs will not be obsoleted but will be marked `[EditorBrowsable(Never)]`.

- The `SerializableAttribute` type
- The `NonSerializedAttribute` type

These changes only affect applications targeting .NET 8+. If a project targets multiple runtimes, these new annotations will be `#if` conditioned to apply only to .NET 8+.

The reason for using two new warning codes is that these warnings are directed at two different audiences. The first warning code (tentatively `SYSLIB0050`) is directed toward people who *invoke* the legacy serialization infrastructure. This consists mostly of people who are writing serializers or of application developers who are doing something so specialized that they require direct calls back in to the serialization infrastructure.

The second warning code (tentatively `SYSLIB0051`) is directed at library authors who extend existing `[Serializable]` types. For example, this warning code would be observed by developers who *both* subclass types like `Exception` or `Dictionary<TKey, TValue>` and extend its serialization ctor. The warning code indicates that this is legacy serialization infrastructure intended only for compatibility, and it should not be adopted by new .NET applications.

Most developers who use `[Serializable]` or who implement `ISerializable` themselves will see no warnings, since those types are not being obsoleted at this time. For developers who see warnings because they call into base implementations' now-obsolete members, self-remediation guidance will be provided.

The following types are *not* being obsoleted at this time: `SerializableAttribute`, `NonSerializedAttribute`, `OnSerializingAttribute`, `OnSerializedAttribute`, `OnDeserializingAttribute`, `OnDeserializingAttribute`, `ISerializable`. This would be too disruptive to the wider ecosystem at this time since other serializers within .NET understand these attributes (but do not honor them correctly). Rather, we must prioritize getting people off of legacy dangerous deserializers by providing better guidance and stepping stones before tackling this longer tail of work. To support that principle, these APIs will not be obsoleted until .NET 9. However, since `SerializableAttribute` and `NonSerializedAttribute` are within the `System` namespace, it's prudent to mark them `[EditorBrowsable(Never)]` as part of this wave.

No new `[Serializable]` types introduced (.NET 8)

In .NET 6, we had stopped attributing new types with `[Serializable]`, carving out an exception only for `Exception`-derived types in projects which targeted *netfx* or *netstandard*.

Starting with .NET 8, we will no longer attribute *any* new types with `[Serializable]`, voiding the previously granted exception. (Already-shipped types will retain their existing annotations.)

Remainder of legacy serialization infrastructure obsoleted (.NET 9)

When targeting .NET 9+, the following types will be marked `[Obsolete]` as *warning*.

- `SerializableAttribute`
- `NonSerializedAttribute`
- `OnSerializingAttribute`
- `OnSerializedAttribute`
- `OnDeserializingAttribute`
- `OnDeserializedAttribute`
- `ISerializable`
- `IDeserializationCallback`

This should represent the balance of unobsoleted APIs which are strongly tied to the legacy serialization infrastructure.

BinaryFormatter infrastructure removed from .NET (.NET 9)

In .NET 9.0, the entirety of the `BinaryFormatter` infrastructure will be removed from the product. Here, "`BinaryFormatter` infrastructure" refers to any API which was obsoleted (as *error*) as part of an earlier stage of this proposal, even if that API is not itself on the `BinaryFormatter` type. The implementation of all such APIs will be to throw `PlatformNotSupportedException`. The internal `BinaryFormatter` implementation will no longer exist in .NET in any form and cannot be reenabled through any compat switch. Functionality may be moved to an unsupported out-of-band package.

All code bases which target *net9.0* or above must be fully migrated away from `BinaryFormatter` at this time. This applies to *dotnet* org first-party code, partner team code, and third-party user code.

There is currently no plan to remove the now-dead APIs themselves from the SDK. Doing so would prevent loading existing libraries which are compiled against those types. However, this may be suggested in the future either concurrent with or occurring after the final "obsolete as error" behavior.

Q\&A

What does this mean for already-deployed applications?

Nothing. There are no behavioral changes being proposed to `BinaryFormatter` for existing in-support versions of .NET, including .NET Framework 2.x - 4.x or .NET Core 2.1 - 3.1. Applications already deployed on these frameworks will continue to work correctly and will enjoy full support for the lifetime of the product. We will not modify the behavior of `BinaryFormatter` as part of any servicing release.

However, per the [BinaryFormatter security guide](#), we recommend that all applications migrate off of `BinaryFormatter` as they find the opportunity to do so.

What serializer should be used instead of `BinaryFormatter` ?

This really depends on the scenario. XML and JSON serializers follow open standards and are often appropriate when the client and the server are written by two different entities. Compressed binary formats may be appropriate for proprietary protocols which want to minimize bandwidth usage. Some serializers opt for speed (total objects serialized or deserialized per second) at the expense of producing slightly larger payloads.

The [BinaryFormatter security guide](#) contains recommendations for choosing a serializer appropriate for the given scenario.

Is `BinaryFormatter` officially supported in .NET 5.0 and beyond?

Yes, per the limitations described in this document. As long as the type exists in the framework and has an implementation, support is available. For example, if an app which uses `BinaryFormatter` behaves as expected in .NET 5.0 but fails after a .NET 5.0.x patch is applied, this regression should be reported to the product team for resolution.

See the earlier section titled "All outstanding `BinaryFormatter`-related issues resolved won't fix" for more information on what categories of issues qualify for support.

Once `BinaryFormatter` is removed from the product (tentatively scheduled for .NET 9), it will no longer be supported by the product team in that release and in subsequent releases. For .NET 7, .NET Framework 4.x, and other runtimes which bundle `BinaryFormatter` as part of the product, the functionality will continue to be supported throughout the normal support period of the .NET runtime itself, subject to the above conditions.

What about other `BinaryFormatter`-equivalent types like `LosFormatter` , `ObjectStateFormatter` , `SoapFormatter` , and `NetDataContractSerializer` ?

These types exist only in .NET Framework 2.x - 4.x. They do not exist in .NET Core or .NET 5.0+. These types will not be reintroduced in any future version of .NET. Applications which wish to migrate to .NET 5.0+ are required to move off of these types. Applications which continue targeting .NET Framework 2.x - 4.x are advised to move off of these types at their earliest opportunity. See the [BinaryFormatter security guide](#) for more information.

Why not make `BinaryFormatter` safe for untrusted payloads?

The `BinaryFormatter` protocol works by specifying the values of an object's raw instance fields. In other words, the entire point of `BinaryFormatter` is to bypass an object's typical constructor and to use private reflection to set the instance fields to the contents that came in over the wire. Bypassing the constructor in this fashion means that the object

cannot perform any validation or otherwise guarantee that its internal invariants are satisfied. One consequence of this is that `BinaryFormatter` is unsafe even for seemingly innocuous types such as `Exception` or `List<T>` or `Dictionary<TKey, TValue>`, regardless of the actual types of *T*, *TKey*, or *TValue*. Restricting deserialization to a list of allowed types will not resolve this issue.

This behavior is intrinsic to `BinaryFormatter` and cannot practically be addressed without changing both the serialized payload format itself and its supported capability set. And once we do that, what we have is essentially a brand new serializer incompatible with `BinaryFormatter`. This would not help existing consumers of `BinaryFormatter`.

Archived from <https://github.com/dotnet/designs/blob/main/accepted/2020/better-obsoletion/binaryformatter-obsoletion.md>. Rendered offline from the local Markdown copy.